ĐÁP ÁN

BÀI 1

Bài 1.1, 1.2, 1.3 xem phần lý thuyết của bài.

1.4. * Giải thuật tính đệ quy tính giai thừa của n với n >= 0 như sau:

```
Giải thuật GIAI_THUAT:
Dữ liệu vào: int n;
Điều kiện áp dụng: n >= 0;
Kết quả trả về: giá trị của n!
{
    if(n == 0) return 1;
    else  return n*GIAI_THUA(n - 1);
}
```

* Giải thuật không đệ quy tính giai thừa của n với n >= 0 như sau:

```
Giải thuật GIAI_THUA
Dữ liệu đầu vào: int n;
Điều kiện áp dụng: n >= 0;
Kết quả trả về: n!;
int i,x;
if (n == 0) return 1;
else
{
        i = 1;
        x = 1;
        while(i <= n)
        {
                x *= i;
                i++;
        }
      return x;
}
```

```
Giải thuật USCLN
Dữ liệu đầu vào: int a,b;
Điều kiện áp dụng: a >= 0 và b >= 0
Kết quả trả về: ước số chung lớn nhất của a và b
{
        if((a == 0)||(b == 0) return (a + b);
        else if(a == b) return a;
        else if(a > b) return UCLN(a - b, b);
        else return UCLN(a, b - a);
}
```

1.5. * Giải thuật đệ quy tìm ước số chung lớn nhất của hai số nguyên:

AUM VIỆT NAM

* Giải thuật không đệ quy tìm ước số chung lớn nhất của hai số nguyên:

```
Giải thuật USCLN
Dữ liệu đầu vào: int a,b
Điều kiện áp dụng: a >= 0 và b >= 0
Kết quả trả về: ước số chung lớn nhất của a và b
{
        if((a == 0)||(b == 0)) return (a+b);
        while(a!= b)
        {
                if(a > b)  a = a - b;
                else b = b - a;
}return a;
}
```

BÀI 2

2.1. Khai báo cấu trúc dữ liệu của danh sách liên kết đôi như sau:

```
#define Max_Size N
typedef Kieu_du_lieu ElementType;
typedef struct NodeType
  {
    ElementType Data;
    struct NodeType *next,*prev;
  }Node;//Cấu trúc của 1 nút
typedef struct List
  {
    NodeType *phead,*ptail;
  }DLIST; //Cài đặt structure để quản lí DLIST
```

2.2. Với khai báo về danh sách liên kết đơn bằng mảng (như trong bài) thủ tục đó được xây dựng như

AUM VIỆT NAM

```
void Nhap(NodeType *Node[Max_Size],NodeType *List,int n)
{
   NodeType *TempPtr;
   int cout,i;
   cout = 0;
   i = 0;
   while (cout <= n)
    {
         cout++;
         printf("nhap du lieu nut thu %d la:Node[%d]="i,i);
         scanf("&x",Node[i] -> Data);
         TempPtr -> Data = Node[i] -> Data;
         if(cout == n)
                     TempPtr -> Link = NULL;
         else
```

AUM VIỆT NAM

```
                    TempPtr -> Link = Node[i + 1];
       i++;
     }

    List = Node[0];
}
```

2.3.

Đây chính là thao tác chèn một phần tử vào một danh sách liên kết đã có thứ tự để được danh sách không thay đổi thứ tự sắp xếp các phần tử. Trước khi chèn một phần tử vào danh sách liên kết, ta phải định vị trí của nó trong danh sách.

Thủ tục Nhap() để nhập vào một dãy số nguyên và lưu nó trong danh sách liên kết theo thứ tự không giảm được xây dựng như sau:

```
void Nhap(NodeType *Node[Max_Size],NodeType *List,int n)
{
  NodeType *TempPtr;
  int cout,i;
  cout = 0;
  i = 0;
  while (cout <= n)
   {
        cout++;
        printf("nhap du lieu nut thu %d la:Node[%d]=",i,i);
        scanf("&x",Node[i] -> Data);
        TempPtr -> Data = Node[i] -> Data;
        if(cout == n)
                    TempPtr -> Link = NULL;
        else
                    TempPtr -> Link = Node[i + 1];
        i++;
   }
  //doan nay sap xep cac nut vua nhap vao theo thu tu tang dan
  int k,j,min;
  ElementType tg;
  for (k = 0; k < n - 2; k++)
   {
        min = k;
        for (j = i + 1; j < n - 1; j++)
             if (Node[min] -> Data > Node[j] -> Data)
                                                  min = j;
        tg = Node[k] -> Data;
        Node[k] -> Data = Node[min] -> Data;
        Node[min] -> Data = tg;
    }
  List = Node[0];
}
```

AUM VIỆT NAM

2.4.a. Khai báo cấu trúc dữ liệu để lưu một đa thức trên máy tính

```
#define Max 50
struct PolyNode
        {
                float Coef;
                int Expo;
                struct PolyNode *Link;
        } Poly[Max];
```

b. Thủ tục nhập thông tin của đa thức

```
void Init(PolyNode *P[Max],int n)
{
     //luu he so cua da thuc bac n  theo chieu mu tang dan
     int i = 0;
     while((i <= n)&&(P[i] -> Expo == n))
      {
             printf("nhap he so thu %d:", i);
             scanf("&f",P[i] -> Coef);
             printf("nhap so mu thu %d:", i);
             scanf("&d",P[i] -> Expo);
             if (i == n )
                                 P[i] -> Link = NULL;
             else
                                 P[i] -> Link = P[i + 1];
             i++;

      }
     //xep lai mang luu he so theo chieu mu giam dan
      int k,j,max,tg;
      for (k = 0; k < n - 1; k++)
          {
             max = k;
             for (j = i + 1; j < n; j++)
                    if (P[max] -> Expo < P[j] -> Expo)
                                          max = j;
             tg = P[k] -> Expo;
             P[k] -> Expo = P[max] -> Expo;
             P[max] -> Expo = tg;
          }
}
```

c. Thủ tục cài đặt phép cộng đa thức

```
PolyNode *PolyAdd(PolyNode *A[Max],PolyNode *B[Max])
{
   PolyNode *C[Max];
```

AUM VIỆT NAM

```
if(A[0] -> Expo > B[0] -> Expo)
        {
          int i; i =
          0;
              while (A[i] -> Link != NULL)
                    {
                       int j; i =
                       0;
                       int found;
                       found = 0;
                       while (B[j] -> Link != NULL)
                                {
                                   if (A[i] -> Expo == B[j] -> Expo)
                                                                found = 1;
                                    j++;
                                }
                           if (found == 1)
                                    while (B[j] -> Link != NULL)
                                    {
                                                if (A[i] -> Expo == B[j] -> Expo)
                                                {
                                                  C[i]->Coef=A[i]->Coef+B[j]->Coef;C[i] -> Expo =
                                                  A[i]-> Expo;
                                                }
                                            j++;
                                    }
                                else
                                    {
                                            C[i] -> Coef = A[i] -> Coef;
                                            C[i] -> Expo = A[i] -> Expo;
                                    }
                           i++;
                    }
              }
          else
              {
                    int i; i =
                    0;
                    while (B[i] -> Link != NULL)
                       {
                                int j; i =
                                0;
                                int found;
                                found = 0;
                                while (A[j] -> Link != NULL)
                                        {
```

AUM VIỆT NAM

```
                                    if (B[i] -> Expo == A[j] -> Expo)
                                                found = 1;
                                     j++;
                                   }
                            if (found == 1)
                                  while (A[j] -> Link != NULL)
                                   {
                                     if (B[i] -> Expo == A[j] -> Expo)
                                      {
                                          C[i] -> Coef = B[i]-> Coef +
A[j] -> Coef;
                                          C[i] -> Expo = B[i] -> Expo;
                                      }
                                     j++;
                                   }
                            else
                             {
                                  C[i] -> Coef = B[i] -> Coef;
                                  C[i] -> Expo = B[i] -> Expo;
                             }
                            i++;
                        }
                    }
    return C[Max];
}
```

BÀI 3

3.1.

a. 7.3333 b. 2 c. 5.6 d. 55

3.2.

a. A B + C D - *
b. A B C / + D +
c. A B + C / D +
d. A B C D E - - - -
e. A B and C or
d. A 3 > A 9 > and A 0 > not or

3.3. Chương trình dùng một ngăn xếp để đảo ngược một số nguyên dương.

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#define max 100
//cai dat stack dung mang
typedef struct
        {
        int top_id;
      int Element[max];
```

AUM VIỆT NAM

```
    }Stack;
//khoi tao stack
void StackInit(Stack *S)
            {
            S -> top_id = -1;
          return;
    }
//kiem tra stack rong int
StackEmpty(Stack *S)
            {
            return (S -> top_id == -1);
    }
//bo sung 1 phan tu vao stack void
PUSH(Stack *S,int X)
            {
            if(S -> top_id == max - 1)
          {
            printf("ngan xep day"); return;
          }
          else
          {
            S -> top_id++;
              S -> Element[S -> top_id] = X; return;
          }
    }
//loai bo 1 phan tu khoi dinh cua stack int POP(Stack
*S)
            {
            if(StackEmpty(S))
            {
                    printf ("ngan xep rong\n"); return
                NULL;
            }
           else
            {
                    return S -> Element[S -> top_id--];
              }
    }
void main(void)
{
            clrscr();
    fflush(stdin); int
    A[max]; Stack *S;
    StackInit(S);
```

AUM VIỆT NAM

```
   int i,n,j,p,ch;
   while(1)
   {
        printf("\n1.nhap so nguyen duong\n2.dao nguoc so nguyen
duong\n3.exit\n");
        printf("chon 1 chuc nang: ");
        scanf("%d",&ch);
      switch(ch)
      {
        case 1:         do
                                          {
                                                  printf("nhap so nguyen
duong n =");
                                          scanf("%d",&n);
                                          }while(n <= 0);
                                          i = 0;
                  while(n>0)
                                          {
                                          A[i] = n%10;
                                          i++;
                                          n = n/10;
                                          }
                                          for (j = i - 1; j >= 0;j--)
                                          {
                                                  PUSH(S,A[j]);
                  }
                  getch();                        <<
                  break;
         case 2:  printf("so dao nguoc cua so nguyen duong: ");
                                          while(!StackEmpty(S))
                                          {
                                          p = POP(S);
                                          printf("%d",p);
                                          }
                  getch();
                  break;
         case 3:        exit(0);break;
      }
   }

   getch();
}
```

3.4. Chương trình đổi một số từ hệ cơ số 10 sang hệ cơ số 2 như sau:

```
#include <iostream.h>
#include <stdio.h>
#include <conio.h>
```

AUM VIỆT NAM

```
#define Max 100
typedef int ElementType;
typedef struct{
   ElementType Elements[Max];
   int Top_idx;
}Stack;

/*Khoi tao 1 ngan xep rong*/ void
StackInit(Stack *S){
   S -> Top_idx = Max;
}

/*Kiem tra ngan xep co rong khong*/ int
StackEmpty(Stack S){
   return (S.Top_idx == Max);
}

/*Kiem tra ngan xep co day khong*/ int
StackFull(Stack S){
   return (S.Top_idx == 0);
}
/*Tra lai noi dung phan tu tren dinh*/
ElementType Top(Stack S){
   if(!StackEmpty(S))
      return S.Elements[S.Top_idx]; else
   printf("Stack rong");
}
/*Xoa 1 phan tu khoi ngan xep*/ void
Pop(Stack *S){
   if(!StackEmpty(*S))
      S -> Top_idx = S -> Top_idx + 1; else
   printf("Stack rong");
}

/*Day 1 phan tu vao ngan xep*/  void
Push(ElementType x, Stack *S){
   if(StackFull(*S))
```

AUM VIỆT NAM

```
        printf("Stack day");
    else{
        S -> Top_idx = S -> Top_idx - 1; S ->
        Elements[S -> Top_idx] = x;
    }

}
```

```
/*Tim va in ra so nhi phan tuong ung*/
void print_binary(int n){
  Stack S;
  StackInit(&S);
  while(n != 0){
         Push(n%2,&S);
         n = n/2;
  };
    printf("So nhi phan tuong ung la:");
  while(!StackEmpty(S)){
    printf("%d",Top(S));
    Pop(&S);
  }
}
/*Chuong trinh chinh*/
void main(){
  int n;
  clrscr();
  fflush(stdin);
  printf("Nhap vao 1 so nguyen:");
  scanf("%d",&n);;
  print_binary(n);
  getch();
}
```

AUM VIỆT NAM

3.5. Chương trình chuyển một biểu thức từ dạng trung tố sang dạng ký pháp Balan.

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <ctype.h>

  //nhap bieu thuc
void Input(char st[])
{
        printf("Nhap vao bieu thuc toan hoc: ");
        gets(st);
}
 //kiem tra toan tu
int LaToanTu(char ch)
{
        return ( (ch=='+') || (ch=='-') || (ch=='*') || (ch=='/') );
}
 //kiem tra toan hang
int LaToanHang(char ch)
{
        return (isalpha(ch)) || (isdigit(ch) );
}
```

AUM VIỆT NAM

```
 //kiem tra do uu tien phep toan int
UuTien(char ToanHang)
{
        switch (ToanHang)
        {
                case '+': return 1; case '-':
                return 2; case '/': return 3;
                case '*': return 4;
        }
        return 0;
}
//chuyen doi dang trung to ve hau to
//dung 2 stack: 1- Stack: luu toan tu. 2-Result: luu ket qua chuyen ve hau to.
char *ConvertToPostFix(char st[])
{
        int i = 0;
    int Stack_top = -1;//khoi tao ngan xep Stack luu toan tu va dau '('
    int Result_top = -1;//khoi tao ngan xep Result luu toan hang char
        Stack[50],Result[50];

        while(i < strlen(st))
        {
                if (st[i]=='(')                         //Neu gap dau (
                        Stack[++Stack_top] = st[i];     //Push dau ( vao Stack

                else if (LaToanHang(st[i])) //Neu la Toan hang Result[++Result_top] = st[i];     //Dua vao chuoi Ket qua

                else if (LaToanTu(st[i]))               //Neu la Toan tu
                {
                        while (LaToanTu(Stack[Stack_top]) && (UuTien(Stack[Stack_top]) >= UuTien(st[i])))
                        //Trong khi Toan tu o dinh Stack co do uu tien lon hon Toan tu dang xet thi day ra vao Result
                                Result[++Result_top] = Stack[Stack_top--];
                        Stack[++Stack_top]=st[i];       //Them Toantu dang xet vao Stack
                }
                else if (st[i]==')')                    //Neu gap dau )
                {
                    while (Stack[Stack_top]!='(')
                    Result[++Result_top] = Stack[Stack_top--];
```

AUM VIỆT NAM

```
        //Push cac phan tu o Stack ra day vao Result cho den khi gap
dau '('
                        Stack_top--;
                }

                i++;
        }

        while(Stack_top ! = -1)//Push toan bo cac Toan hang (neu con)
trong Stack ghi vao chuoi Ket qua
                Result[++Result_top] = Stack[Stack_top--];

        Result[++Result_top] = NULL;   //Ket thuc chuoi Ket qua

        return Result;
}
void main()
{
        char st[50];
        clrscr();

        Input(st);
        printf("%s",ConvertToPostFix(st));
        getch();
}
```

BÀI 4.

4.1, 4.2. Xem phần lý thuyết của bài 4.

4.3. Dùng thuật toán như được mô tả trong phần ứng dụng của ngăn xếp, tìm đường đi ngắn nhất từ đỉnh 1 đến đỉnh 9 như sau:

Thao tác	Kết quả Q
Xếp 1 vào hàng đợi	1
Lấy 1 phần tử ra khỏi hàng đợi: 1	rỗng
Xếp tất cả các phần tử kề với 1 vào Q	2, 3, 5, 6
Lấy 1 phần tử ra khỏi hàng đợi: 2	3, 5, 6
Xếp tất cả các phần tử kề với 2 vào Q	3, 5, 6, 9 (1 đã được xếp nên không xếp vào)

Như vậy sau 5 thao tác ta đã tìm được phần tử có giá trị bằng 9. Lần ngược lại các thao tác, ta sẽ thu được 1 đường đi ngắn nhất từ 2 đến 7:

9 được xếp vào Q do liền kề với 2

2 được xếp vào Q do liền kề với 1

Vậy đường đi ngắn nhất từ 1 đến 9 tìm được là: 1 -> 2-> 9

VIỆT NAM

4.4. Chương trình chuyển từ hệ thập phân sang hệ nhị phân

```
#include<conio.h>
#include<stdio.h> #define
Max 50 typedef int
ELEMENT; struct
            LIST_ELEME
NT
  {
            ELEMENT info;
            struct LIST_ELEMENT *next;
  };
 struct QUEUE_LIST
 {
          struct LIST_ELEMENT *H;
          int S;
 };
//ham khoi tao
QUEUE_LIST INITIALIZE( QUEUE_LIST q)
{
 q.H = NULL;
 q.S = 0;
 return q;
}
//ham kiem tra hang doi co rong khong
int IS_EMPTY(QUEUE_LIST q)
//q da duoc khoi tao bang ham INITIALIZE(QUEUE_LIST q)
{
   int empty;
   empty = 0;
   if (q.S == 0)
                    empty = 1;
   else
                    empty = 0;
   return
                    empty;
}
//ham kiem tra hang doi co day khong
int IS_FULL(QUEUE_LIST q)
//q da duoc khoi tao bang ham INITIALIZE(QUEUE_LIST q)
{
   int full; full
   = 0;
   if (q.S == Max)
                    full = 1;
```

AUM VIỆT NAM

```
   else
                        full = 0;
   return
                        full;
}
QUEUE_LIST ENQUEUE(QUEUE_LIST q,ELEMENT e)
{
   LIST_ELEMENT  *p,*m;
   m = new LIST_ELEMENT;
   p = new LIST_ELEMENT;
   m -> info = e;
   m -> next = NULL;
   if (q.S == 0)
                        q.H = m;
   else
                        {
                           p = q.H;
                           for (int i = 0; i < q.S - 2; i++) p = p -> next;
                           p -> next = m;
                        }
         q.S = q.S + 1;
   return q;

}
ELEMENT DEQUEUE(QUEUE_LIST q)
{
   ELEMENT e;
   if (q.S == 0)
                        printf("hang doi rong khong the lay phan tu");
   else
                {
                           e = q.H -> info;
                           q.S = q.S - 1;
                }          q.H = q.H -> next;
      return e;
}
void main()
{
clrscr(); int
N,m;
//ELEMENT a[Max]; printf("nhap
vao so N ="); scanf ("%d",&N);
```

AUM VIỆT NAM

```
QUEUE_LIST Q;
while (N != 0)
{
        m = N % 2;
        Q = ENQUEUE(Q,m);
        N = N / 2;
}
printf("%d",Q.S);
printf("\nBieu dien so dang co so 2 la: ");
for (int tg = 0; tg <= Q.S - 1; tg++)
        {
          a[tg] = DEQUEUE(Q);
        }
for (int k = Q.S - 1; k >= 0; k--)
        printf("%d",a[tg]);
getch();
}
```

BÀI 5.

5.1. Xem phần lý thuyết của bài.

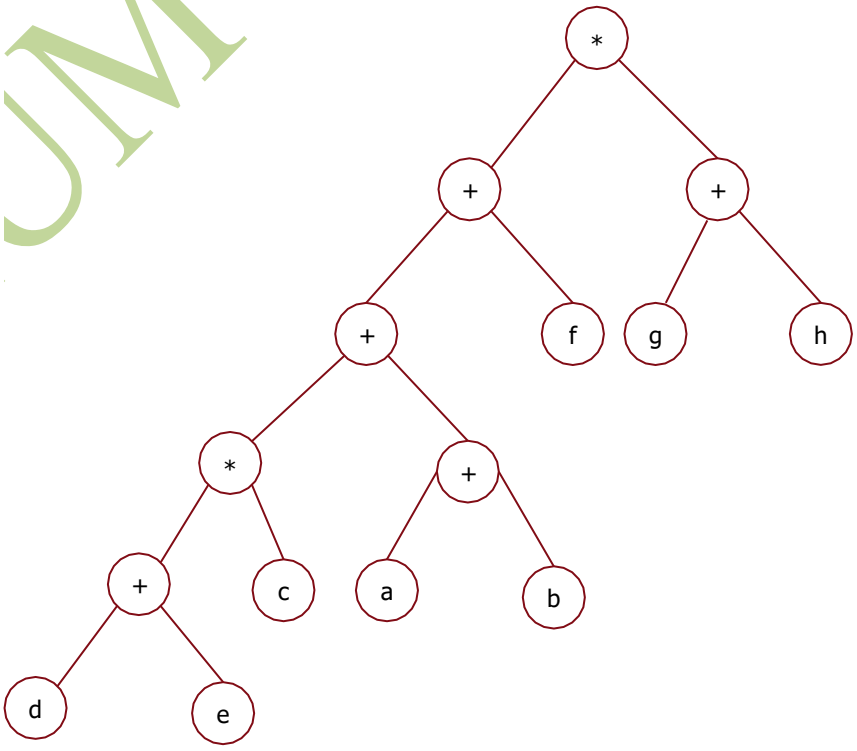
5.2. – Biểu thức duyệt tiền tự: A B D E I M N C F G J K H L

– Biểu thức duyệt trung tự: D B M I N E A F C J K G H L

– Biểu thức duyệt kiểu hậu tự: D M N I E B F J K G L H C A

5.3. A B C D E F G H I J K L M N

5.4.



UM VIỆT NAM

5.5.

a. Kết quả của các phép duyệt:

– Tiền tự: A B D H G C F H I J

– Trung tự: D B G E A C H F I J

– Hậu tự: D G E B H J I F C A

b. Dùng cấu trúc dữ liệu mảng để lưu trữ cây nhị phân này như sau:

	infor	left	right
1	A	2	3
2	B	4	5
3	C	0	6
4	D	0	0
5	E	7	0
6	F	8	9
7	G	0	0
8	H	0	0
9	I	0	10
10	J	0	0

BÀI 6.

6.1. Xem lại phần lý thuyết của bài 6.

6.2. Cài đặt thuật toán tìm kiếm tuần tự bằng sử dụng:

```
–vòng lặp for
int LinearSearch(KeyType X, KeyType *R)
{
        int i;
        for(i = 0; i < n; i++)
        {
                if (R[i] == X) return i;
        }
        return -1;
}
```

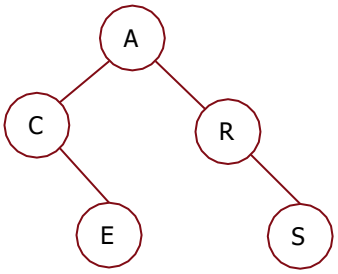
```
-vòng lặp while
int LinearSearch(KeyType X;KeyType *R)
{
        int i;
        i = -1;
        do
        {
                i++;
}while ((R[i] != X)&&(i < n));
return i;

}
```

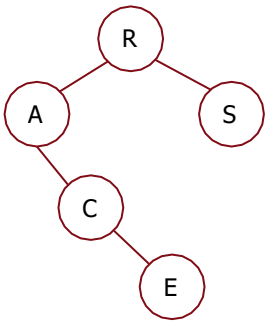
AUM VIỆT NAM

6.3.

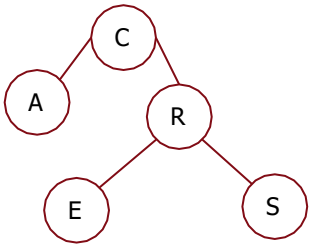
a)



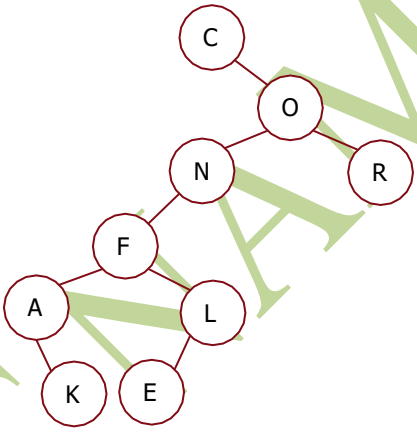
b)



c)



d)



6.4.

```
#define Max 50
typedef int Mang[Max];
void Nhap(Mang A,int n)
{
        int i;
        printf("nhap vao day so nguyen");
        for(i = 0; i < n; i++)
        {
                printf("A[%d]",i);
                scanf("%d",A[i]);
        }
}
void Tim(Mang A,int X,int n)
{
        int i, j;
        Mang KQ;
        i = -1;
        for(i = 0; i < n; i++)
                if(A[i] == X)
                {
                        j++;
                        KQ[j] = i;
             }
        if(j >= 0)
        {
                printf("%d xuat hien tai cac vi tri",X);
                for(i = 0; i <= j; i++)
                        printf("%d",KQ[i]);
```

ÂU VIỆT NAM

```
        }
        else printf("khong tim thay trong day");
}
main(void)
{
        int n,X;
        Mang A;
        printf("nhap so phan tu toi da cua day n=");
        scanf("%d",&n);
        Nhap(A,n);
        printf("Nhap vao phan tu can tim X=");
        scanf("%d",&X);
        Tim(A,X,n);
       getch();
}
```

6.5.

```
#define max 50
typedef int Mang[max];
int Tim(Mang A, int X,int n) //tim vi tri se chen vao
{
        int i;
        i = 0;
        while((A[i] != X)&&(i < n))
        {
                i++;
        }
        return (i);
}

main(void)
{
        int n,i,X,y,k;
        Mang A;
        printf("nhap so phan tu cua day:");
        scanf("%d",&n);
        i = 0;
        while(i < n)
        {
                printf("nhap vao phan tu thu %d la:",i);
                scanf("%d",&X);
        y = Tim(A,X,i);//vi tri se duoc chen
        //thao tac Chen
        for(k = i; k > y; k--)
                A[k] = A[k-1];
        A[y] = X;
        i++;
        }
   for(i = 0; i < n; i++)
   printf("A[%d] = %d\n",i,A[i]);
getch();
}
```

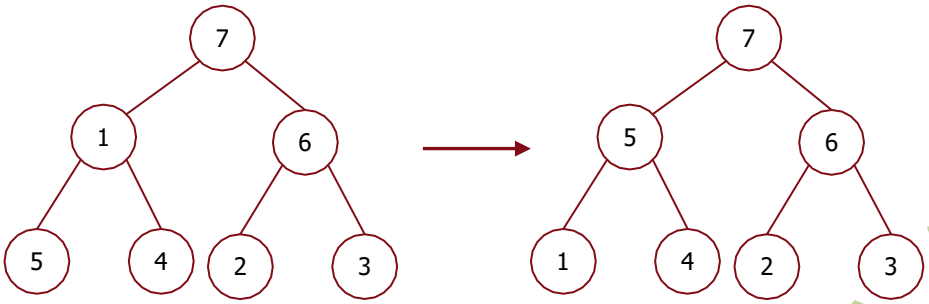
ÂU VIỆT NAM

BÀI 7.

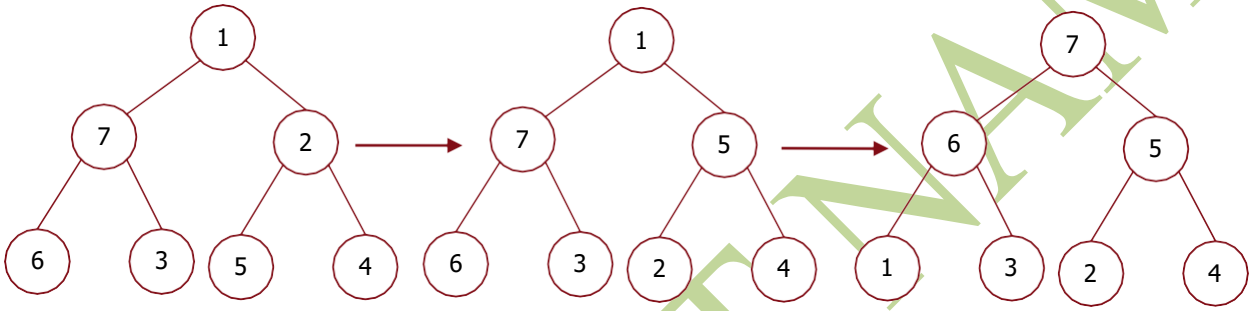
7.1, 7.2. Xem phần lý thuyết của bài học

7.3.

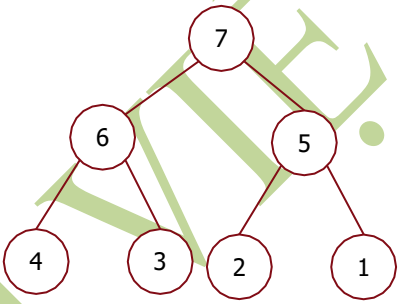
a)

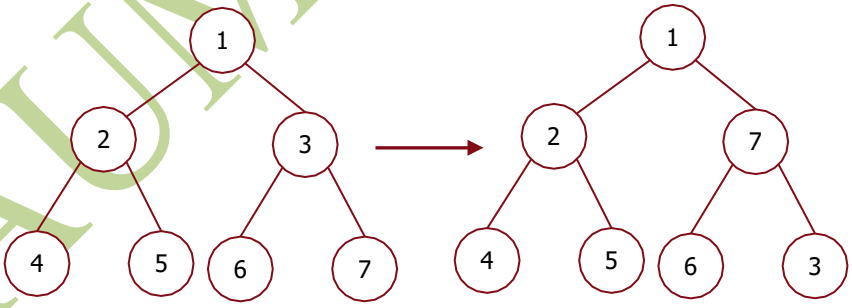


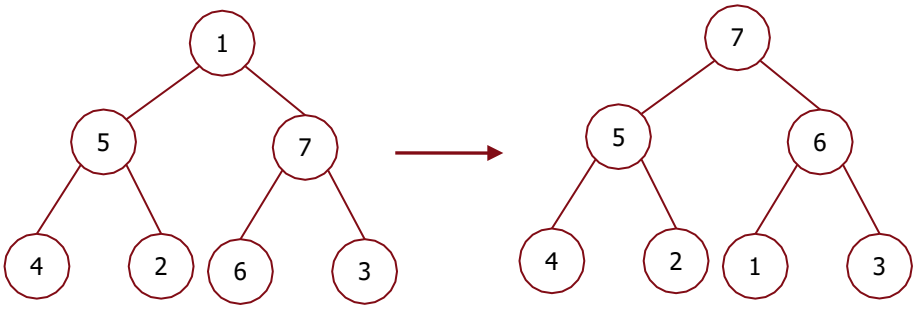
b)



c)







AUM VIỆT NAM

7.4.

```
#include <conio.h>
#include <stdio.h>
#define Max_Size 10
#define HeapLimit 10 typedef
float ElementType; struct
        HeapType
        {
          ElementType Array[HeapLimit];
          int m;
        } Heap;


typedef    int KeyType;
struct KeyArray
                {
                        KeyType Array[Max_Size];
                        int n;
                };
KeyArray a;
void Sortinsert(); void
SortSelect(); void
Sortbubble(); void
NhapData(); void
main()
{
        int luachon;
        do
        {
          printf("                                    MENU CUA CHUONG TRINH");

          printf("\n º 1.Nhap vao mot day so nguyen "); printf("\n º 2.Sap xep
          kieu chen - Insertion sort"); printf("\n º 3.Sap xep kieu chon -
          Selection sort"); printf("\n º 4.Sap xep kieu doi cho");  printf("\n");
          printf("\n          Ban hay chon mot so:");
          scanf("%d",&luachon);
          switch (luachon)
          {
                case 1:NhapData();break; case
                2:Sortinsert();break; case
                3:SortSelect();break; case
                4:Sortbubble();break;

          }
        }
    while (1);
```

AUM VIỆT NAM

```
}
void NhapData()
{
   a.n = 0;
   while(a.n < Max_Size - 3)
   {

            printf("Array[%d]=",a.n);
            scanf("%d",&a.Array[a.n]); a.n++;
   }

}

void Sortinsert()
{

            int i,j;
            KeyType x;
        i = 1;
            while ( i <= a.n - 1 )
                  {
                              x = a.Array[i] ; j = i - 1;
                              while (( j >= 0 )&&(x < a.Array[j]))
                                                {
                                                            a.Array[j + 1] =
a.Array[j];
                                                            j--;
                                                }
                              a.Array[j + 1] = x ; i++;

                  }
            for (int k = 0; k < a.n; k++)
  printf("Array[%d] = %d\n",k,a.Array[k]);
}

void SortSelect()
{
   int i,j,min;
   KeyType tg;
            for (i = 0; i < a.n - 2; i++)
                        {
                              min = i;
                              for (j = i + 1; j < a.n - 1; j++)
                                          if (a.Array[min] > a.Array[j])

min = j;
                              tg = a.Array[i]; a.Array[i] =
                              a.Array[min]; a.Array[min] = tg;
```

AUM VIET NAM

```
                }
  for (int k = 0; k < a.n; k++)
  printf("Array[%d] = %d\n",k,a.Array[k]);

}
void Sortbubble()
{
  int i,j;
  KeyType tg;
  for (i = 0; i < a.n - 2; i++)
                for (j = a.n - 1; j >= i + 1; j--)
                              if (a.Array[j - 1] > a.Array[j])
                                            {
                                                          tg = a.Array[j - 1];
                                                          a.Array[j-1] = a.Array[j];
                                                          a.Array[j] = tg;
                                            }
  for (int k = 0; k < a.n; k++)
  printf("Array[%d] = %d\n",k,a.Array[k]);

}


  getch();
```

CÂU HỎI THƯỜNG GẶP

1. Cấu trúc dữ liệu và giải thuật là gì?

Trả lời:

Cấu trúc dữ liệu là cách thức tổ chức sắp xếp dữ liệu để biểu diễn thông tin trên máy tính. Dữ liệu có thể được lưu trữ tại bộ nhớ trong để trực tiếp xử lý, hoặc được lưu trong các thiết bị lưu trữ ngoài ví dụ như dưới dạng tệp dữ liệu trên ổ cứng.

Giải thuật là các phương pháp giải quyết vấn đề phù hợp cho việc triển khai trên máy tính. Trong khuôn khổ của giáo trình này, giải thuật được hiểu là các biện pháp, cách thức để thao tác dữ liệu.

Cấu trúc dữ liệu và giải thuật là hai thuật ngữ luôn đi kèm với nhau. Cấu trúc dữ liệu là đầu vào và cũng là đầu ra của giải thuật. Với mỗi một loại cấu trúc dữ liệu cụ thể phải có những giải thuật phù hợp. Ngược lại, mỗi giải thuật chỉ thực sự hiệu quả khi được áp dụng trên một số cấu trúc dữ liệu nhất định.

2. Kiểu dữ liệu là gì? Trong nội dung môn học này có những kiểu dữ liệu nào?

Trả lời:

Như chúng ta đã biết, máy tính biểu diễn dữ liệu dựa trên các số nhị phân (chỉ nhận hai giá trị 0 hoặc 1). Trường giá trị có thể biểu diễn là tập hợp các giá trị rời rạc, không thích hợp cho việc biểu diễn trực tiếp các giá trị trong thực tế thường là các giá trị liên tục. Vì vậy, kiểu dữ liệu được dùng để biểu diễn thông tin cần phải sát với kiểu giá trị của các thông tin đó trong thực tế. Kiểu dữ liệu được chia thành 2 loại: Kiểu dữ liệu cơ bản và Kiểu dữ liệu trừu tượng.

ỦY VIỆT NAM

- Kiểu dữ liệu cơ bản: là các kiểu dữ liệu có sẵn trên hầu hết các máy tính, và được hỗ trợ trong hầu hết các ngôn ngữ lập trình. Chúng bao gồm kiểu dữ liệu số nguyên (Integer), kiểu dữ liệu số thực (Real), kiểu dữ liệu giá trị logic (Boolean), và kiểu dữ liệu ký tự (Char). Ngoài ra còn một kiểu dữ liệu nữa hay được sử dụng trong các thuật toán là kiểu dữ liệu con trỏ (Pointer).
- Kiểu dữ liệu trừu tượng: là kiểu dữ liệu do người dùng tự định nghĩa dựa trên các kiểu dữ liệu cơ bản sẵn có. Những kiểu dữ liệu mới được định nghĩa đó được gọi là các kiểu dữ liệu trừu tượng, để phân biệt với các kiểu dữ liệu cơ bản. Chúng bao gồm dữ liệu kiểu bản ghi (Record), dữ liệu kiểu mảng (Array).

3. Ngôn ngữ diễn đạt giải thuật là gì? Có quy chuẩn nào cho việc sử dụng ngôn ngữ diễn đạt giải thuật không?

Trả lời:

Cũng giống như với cấu trúc dữ liệu, với mục đích thiết kế chương trình không phụ thuộc vào một máy tính hay một ngôn ngữ nhất định, các giải thuật cũng phải được diễn đạt hay biểu diễn bằng một ngôn ngữ chung. Ngôn ngữ đó vừa phải mang tính trừu tượng giúp cho người dùng không cần quan tâm tới những đặc tả của máy tính hay các câu lệnh cụ thể của bất kỳ ngôn ngữ nào. Hơn nữa, ngôn ngữ diễn đạt giải thuật càng gần với ngôn ngữ tự nhiên thì càng dễ cho người đọc nắm bắt các giải thuật được viết. Nhưng ngôn ngữ diễn đạt giải thuật cũng phải càng gần với các ngôn ngữ lập trình hiện có càng tốt để giúp cho người triển khai phần mềm dễ dàng chuyển từ thiết kế của chương trình sang một triển khai trên một ngôn ngữ lập trình cụ thể được lựa chọn.

Nói chung, không có một quy chuẩn nhất định cho ngôn ngữ diễn đạt phần mềm. Vì thế tồn tại rất nhiều phiên bản của ngôn ngữ diễn đạt phần mềm. Có những phiên bản được xây dựng ít nhiều dựa trên một ngôn ngữ lập trình cụ thể mà nhiều người biết tới. Có thể kể ở đây như ngôn ngữ lập trình Pascal hay ngôn ngữ lập trình C.

4. Trong khai báo biến, có quy định nào cho việc đặt tên biến không?

Trả lời:

Mặc dù không bắt buộc, nhưng để tiện cho việc đọc giải thuật, tên biến thường được viết bằng chữ thường, còn tên kiểu dữ liệu thường được viết bằng chữ cái in hoa.

Ví dụ để khai báo một biến (Variable) để sử dụng trong giải thuật, chúng ta dùng cấu trúc:

Kiểu_dữ_liệu: tên_biến; Type: Variable;

5. Ưu điểm lớn nhất của việc thiết kế giải thuật theo phương pháp đệ quy là gì?

Trả lời:

Ưu điểm lớn nhất của phương pháp đệ quy là tính hiệu quả trong thiết kế và lập trình. Như đã thấy trong ví dụ trên, với việc sử dụng lời đệ quy, logic của chương trình được biểu diễn hết sức đơn giản và trong sáng. Giải thuật đệ quy đặc biệt thích hợp để giải quyết các vấn đề, để triển khai các hàm hay để xử lý các dữ liệu vốn dĩ đã được định nghĩa bằng phương pháp truy hồi.

Tuy nhiên giải thuật đệ quy không thật sự hiệu quả nếu xét về mặt thực hiện chương trình.

6. Phương pháp quay lui là gì?

Trả lời:

Một trong những phương pháp để giải quyết các vấn đề chung chung đó là phương pháp thử và quay lại nếu sai (Trial and Error). Vấn đề được chia nhỏ thành các vấn đề con nhỏ hơn,

và chương trình sẽ chọn thử lần lượt các giá trị có thể của một vấn đề con. Với mỗi giá trị có thể đó chương trình sẽ tìm cách kiểm tra xem có tồn tại lời giải với các vấn đề con còn lại hay không. Nếu không tồn tại lời giải cho các vấn đề con còn lại, chương trình sẽ quay lại thử giá trị có thể tiếp theo của vấn đề con hiện thời. Một vấn đề thường được chia nhỏ thành nhiều tầng theo hướng đệ quy. Việc tìm lời giải cho tầng dưới sẽ được thực hiện bằng một lời gọi đệ quy. Nếu không tìm được lời giải cho tầng dưới thì quay lui lên tầng trên để thử lời giải tiếp theo. Vì vậy, phương pháp này được gọi là phương pháp quay lui.

7. Định nghĩa danh sách con của một danh sách?

Trả lời:

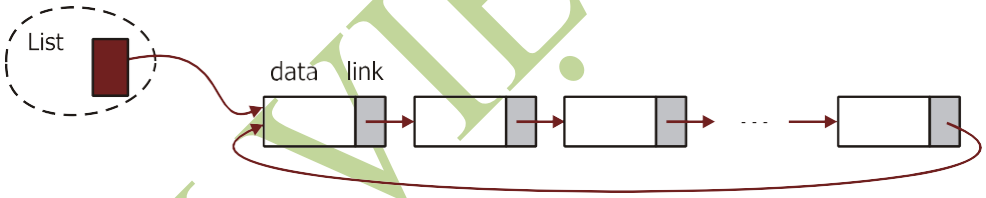
Nếu L = (a_1, a_2, ..., a_n) là một danh sách và i, j là các vị trí, 1 □ i □ j □ n thì danh sách L' gồm tất cả các phần tử từ ai đến aj của danh sách L được gọi là danh sách con của L. Danh sách rỗng được xem là danh sách con của một danh sách bất kỳ.

Danh sách con bất kỳ gồm các phần tử bắt đầu từ phần tử đầu tiên của danh sách L được gọi là *phần đầu* của danh sách L. *Phần cuối* của danh sách L là một danh sách con bất kỳ kết thúc ở phần tử cuối cùng của danh sách L.

8. Danh sách vòng đơn là gì?

Trả lời:

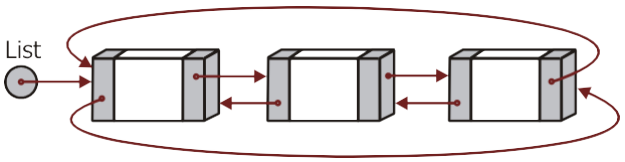
Trong danh sách liên kết đơn, trường liên kết của nút cuối cùng được gán một giá trị đặc biệt. Nếu ta cho trường liên kết của nút cuối cùng trỏ thẳng về nút đầu tiên của danh sách thì khi đó ta sẽ được một kiểu danh sách liên kết mới gọi là danh sách liên kết vòng đơn. Cấu trúc của danh sách liên kết vòng đơn được mô tả bằng hình dưới đây.



9. Danh sách vòng kép là gì?

Trả lời:

Danh sách liên kết vòng kép là danh sách liên liên kết kép trong trường hợp con trỏ Prev của nút đầu tiên trong danh sách trỏ thẳng đến nút cuối cùng trong danh sách và con trỏ Link của nút cuối cùng trong danh sách đó chỉ thẳng đến nút đầu tiên của danh sách. Cấu trúc của danh sách liên kết vòng kép được mô tả trong hình dưới đây:



10. Những bước cơ bản nào để cài đặt danh sách liên kết?

Trả lời:

Để cài đặt được danh sách liên kết, chúng ta phải ít nhất thực được các công việc sau:

- Dùng một phương tiện nào đó để chia bộ nhớ thành các nút, mỗi nút có thành phần dữ liệu và phần liên kết và một cách đặt cho các con trỏ.

AUM VIỆT NAM

- Các thao tác để truy cập tới những giá trị lưu trong mỗi nút.
- Dùng một phương tiện nào để ghi dấu các nút đang dùng và các nút tự do, và để truyền các nút giữa hai vùng nói trên.

11. Khi nào nên sử dụng cấu trúc dữ liệu kiểu danh sách liên kết? Những nhược điểm của cấu trúc dữ liệu kiểu danh sách liên kết?

Trả lời:

Những danh sách có độ dài thay đổi và thường xuyên phải thực hiện thao tác chèn hay xóa đi các phần tử nào đó trong danh sách thì việc cài đặt danh sách liên kết là tốt nhất. Tuy nhiên, cách cài đặt này cũng có một số nhược điểm. Một là chỉ có phần tử đầu tiên của danh sách được truy cập trực tiếp, các phần tử khác muốn truy cập được phải đi qua những phần tử đứng trước nó. Hai là phần bộ nhớ bổ xung đòi hỏi trong cài đặt này. Việc cấp phát vùng nhớ không những cho các phần tử của danh sách mà còn cho phần liên kết để nối các phần tử đó.

12. Phân biệt sự khác nhau giữa cấu trúc dữ liệu kiểu ngăn xếp và cấu trúc dữ liệu kiểu danh sách?

Trả lời:

Cũng giống như danh sách, ngăn xếp là cấu trúc dữ liệu tuyến tính. Nó gồm các đối tượng dữ liệu được sắp thứ tự. Nhưng đối với danh sách, các phép toán xen, loại và truy cập có thể thực hiện ở vị trí bất kỳ của danh sách, còn đối với ngăn xếp các phép toán đó chỉ được thực hiện ở một đầu.

13. Nguyên lý quan trọng của cấu trúc dữ liệu kiểu ngăn xếp là gì?

Trả lời:

Đó là *Last In First Out (LIFO)* "vào sau ra trước".

14. Khi cài đặt ngăn xếp bằng mảng thì hạn chế lớn nhất là gì? Cách khắc phục?

Trả lời:

Hạn chế của việc cài đặt ngăn xếp bằng mảng cũng chính là hạn chế của mảng đó là độ dài của ngăn xếp bị giới hạn bằng chiều dài của mảng. Hạn chế này sẽ được giải quyết khi cài đặt ngăn xếp bằng danh sách liên kết.

15. Nguyên lý quan trọng của cấu trúc dữ liệu kiểu hàng đợi là gì?

Trả lời:

Đó là *First-In First-Out (FIFO)* "vào trước ra trước".

16. Khi nào nên cài đặt hàng đợi bằng mảng? Hạn chế là gì?

Trả lời:

Chúng ta chỉ nên cài đặt hàng đợi bằng mảng khi đã biết chính xác số phần tử tối đa của hàng đợi và không quá lớn. Việc sử dụng mảng để cài đặt hàng đợi cho phép tận dụng các ưu điểm của mảng đó là thời gian truy xuất nhanh vì các phần tử của mảng được truy xuất một cách trực tiếp. Tuy nhiên, hạn chế của việc sử dụng mảng để cài đặt hàng đợi là số phần tử của mảng là cố định, vì vậy kích thước của hàng đợi cũng bị giới hạn (không được vượt quá kích thước của mảng). Hạn chế này sẽ được giải quyết khi cài đặt hàng đợi bằng danh sách liên kết.

AUM VIỆT NAM

17. Mỗi nút được thăm bao nhiêu lần trong phép duyệt cây?

Trả lời:

Phép duyệt cây là phép duyệt các nút trên cây một cách hệ thống sao cho mỗi nút chỉ được thăm một lần. Trong thao tác này chúng ta tìm cách duyệt qua tất cả các nút trên cây để thực hiện một thao tác xử lý nào đó đối với nút (như xem nội dung thành phần dữ liệu).

18. Thế nào là tìm kiếm tuần tự? Ưu nhược điểm của phương pháp này?

Trả lời:

Thuật toán tìm kiếm tuần tự (Sequential Search) là một kỹ thuật tìm kiếm đơn giản. Tư tưởng của thuật toán là: Bắt đầu từ mẩu tin đầu tiên, lần lượt so sánh khóa tìm kiếm với khóa tương ứng của các mẩu tin trong danh sách, cho tới khi tìm thấy mẩu tin có khóa bằng khóa tìm kiếm hoặc đã duyệt hết danh sách mà chưa thấy. Thuật toán tìm kiếm tuần tự dễ thực hiện nhất đối với thông tin lưu trữ dạng mảng. Tuy nhiên trong trường hợp số phần tử của danh sách tìm kiếm là rất lớn, chẳng hạn chúng ta tìm kiếm tên một khách hàng trong danh bạ điện thoại của một thành phố lớn mà theo tìm kiếm tuần tự thì sẽ mất rất nhiều thời gian.

19. Tìm kiếm nhị phân là như thế nào? Ưu nhược điểm của phương pháp này?

Trả lời:

Tư tưởng của thuật toán tìm kiếm nhị phân đó là tìm kiếm dựa trên ứng dụng sơ đồ "chia – để – trị". Điều này có nghĩa là chúng ta sẽ chia những mẩu tin trong danh sách tìm kiếm thành hai phần, xác định xem phần nào chứa khoá cần tìm.

Phép tìm kiếm nhị phân hơn hẳn tìm kiếm tuần tự trên danh sách có thứ tự, tuy nhiên nó cũng đòi hỏi phải được cài đặt tuần tự trên cơ sở mảng để có thể truy cập trực tiếp tới các phần tử của danh sách. Nó không thích hợp khi cài đặt trên danh sách liên kết vì khi đó việc định vị phần tử ở giữa danh sách đòi hỏi phải đi qua danh sách con chứa các phần tử đứng trước phần tử này. Khi đó cho dù là áp dụng thuật toán tìm kiếm nhị phân cũng không mang lại nhiêu hiệu quả như tìm kiếm tuần tự vì vẫn phải duyệt qua từng phần tử.

AUM VIỆT NAM